MyCodeSchool

Code review — py-siruri

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-21 — sha f627497

Ce am verificat: `siruri.py` (fișa 1), `siruri2.py` (fișa 2 + exercițiile tale în plus), `main.py`. Fiecare 🔴 de mai jos l-am **rulat**, nu doar citit.

Pe scurt: ai lucrat mult și ordonat — un commit pe exercițiu, exact cum trebuie. Provocările grele din fișa 2 sunt **toate corecte**: `fara_dubluri`, `cea_mai_lunga_serie`, `este_anagrama`, plus `parola_buna`, `suma_din_text` (și pe `""` !), `litera_din_mijloc`, `formatare_nume`. Tabelele de urmărire din comentarii (`siruri.py:20`, `:52`) sunt foarte bune — continuă așa.

Ce rămâne: **5 funcții din fișe dau alt rezultat decât tabelul** și **3 bug-uri în exercițiile tale în plus**. Aproape toate sunt aceeași greșeală: **valoarea de pornire** sau **cazul pe care nu l-ai încercat**.

🔴 Critice

B1 🔴 **`prima_litera` întoarce tot textul, întors** — `siruri.py:4`

```
prima_litera("Python")  ->  'nohtyP'     (tabelul: 'P')
```

`text[::-1]` e o **felie** cu pasul -1: ia tot șirul, de la coadă la cap. Prima literă e o singură **poziție**, nu o felie. Ai testat doar cu `"a"` — acolo inversul lui `"a"` e tot `"a"`, deci bug-ul s-a ascuns.

💡 Cum ai scris `ultima_litera`, chiar sub ea? Ce poziție are prima literă? 💡 Testează mereu cu un cuvânt mai lung de o literă.

B2 🔴 **`initiale` nu face literele mari** — `siruri.py:48`

```
initiale("ana maria pop")  ->  'amp'     (tabelul: 'AMP')
```

`cuvant[0]` ia litera **așa cum e** în text. Pe `"Luca Todor"` merge pentru că literele erau deja mari — din nou, exemplul ales a ascuns bug-ul.

💡 Ce metodă ai folosit în `prezentare` ca să faci o literă mare?

B3 🔴 **`este_palindrom` ține cont de spații și de literele mari** — `siruri.py:70`

```
este_palindrom("Ele fac cafele")  ->  False   (tabelul: True)
este_palindrom("Ana")             ->  False   (tabelul: True)
```

Pentru Python, `"A"` și `"a"` sunt caractere **diferite**, iar spațiul e și el un caracter. Fișa cere explicit: „nu contează spațiile și nici literele mari". Deci textul trebuie **curățat înainte** de comparație.

💡 Uită-te la primele două rânduri din `este_anagrama` — le-ai scris tu și fac exact curățenia asta.

B4 🔴 `ascunde_cardul` lasă vizibil începutul, nu sfârșitul — `siruri2.py:7`

```
ascunde_cardul("1234567812345678")  ->  '************123456781234'   (tabelul: '************5678')
ascunde_cardul("123")               ->  ''                          (tabelul: '123')
```

`[:-4]` = „totul, **fără** ultimele 4". `[-4:]` = „**doar** ultimele 4". Diferența e de partea cărui `:` stă numărul. La `"123"`, `[:-4]` cere „fără ultimele 4" dintr-un text de 3 → rămâne `""`.

💡 Compară cu `numar_telefon`, chiar sub ea (`siruri2.py:17`) — acolo ai scris felia corect.

B5 🔴 `prezentare` — lipsește „ani" — `siruri2.py:4`

```
prezentare("luca", 15)  ->  'Luca are 15'    (tabelul: 'Luca are 15 ani')
```

Pentru calculator, un singur caracter diferit înseamnă un alt rezultat. Când verifici, compară literă cu literă cu tabelul, nu „seamănă".

B6 🔴 `cel_mai_mare_numar` pornește de la 0 — `siruri2.py:150`

```
cel_mai_mare_numar("-5,-3,-8")  ->  0     (corect: -3)
```

Când maximul pornește de la `0`, niciun număr negativ nu-l poate întrece, iar 0 nici nu e în listă. Funcția întoarce un număr care **nu există** în text. E tiparul din C++: cazul la care nu te-ai gândit.

💡 Uită-te la `cel_mai_mic_numar`, chiar dedesubt — acolo ai pornit corect. De ce? ➕ Același start de la 0: `analiza_vanzari:291`, `analiza_vanzari1:365`, `analiza_prod:396`. Pe datele tale merge, pentru că toate sunt pozitive; regula e aceeași.

B7 🔴 `analiza_note` nu găsește studentul, dacă e primul — `siruri2.py:317-318`

```
analiza_note({"Elena": 5, "Maria": 10, "Andrei": 7})  ->  ('', 5, 1, 7.33)
```

Ai pornit nota minimă de la nota primului student, dar numele de la `""`. Dacă primul **chiar e** cel mai slab, `nota < cea_mai_mica` nu e niciodată adevărat (5 < 5 = fals), deci numele nu se mai completează. Nota și numele trebuie să pornească **de la același student**.

💡 În `analiza_cheltuieli` (`:342–343`) ai pornit corect amândouă — compară cele două funcții.

B8 🔴 **`analiza_vanzari` crapă dacă nu e nicio vânzare mare** — `siruri2.py:300`

```
analiza_vanzari({"Mouse": 100, "Cablu": 50})  ->  ZeroDivisionError: division by zero
```

`contor` numără vânzările ≥ 1500. Dacă nu e niciuna, rămâne 0, iar împărțirea la 0 oprește programul — Python nu dă „liniștit 0", ca C++-ul pe Mac.

💡 În `analiza_tranzactii` (`:257`) te-ai apărat exact de cazul ăsta. De ce acolo da și aici nu?

🟡 Importante

- **M1** `email_valid` (`siruri2.py:207`) cere o **cifră** → `"ana.pop@gmail.com"` iese `False`. Majoritatea email-urilor n-au cifre. E regula ta sau a cerinței?
- **M2 — cheamă, nu rescrie:** `fara_cifre_dublate` (`:416`) e copia identică a lui `fara_dubluri` (`:407`); `cel_mai_lung_cuvant` (`siruri2.py:55`) e copia lui `cuvantul_cel_mai_lung` (`siruri.py:60`). Când două funcții fac același lucru, a doua o **cheamă** pe prima.
- **M3** Variabile cu același nume, definite de mai multe ori în fișier: `vanzari` (`:279` și `:356`), `salarii` (`:263` și `:478`), `numere` (×3). A doua o șterge pe prima — `analiza_vanzari(vanzari)` primește acum orașele, nu produsele. Datele de test stau mai bine în `main.py`.
- **M4** Fișa 2 cerea funcțiile în același `siruri.py`, sub cele din fișa 1. `main.py` afișează o singură funcție — fișa cerea un `print` pentru **fiecare rând** din tabel. Cu `print`-urile alea ai fi prins B1–B5 singur.

🟢 Mărunte

- **C1** `pass` după `return` (`siruri.py:13`, `:18`) — nu se mai execută niciodată, se poate șterge.
- **C2** `if are_cifra == True and ...` → ajunge `if are_cifra and ...`.
- **C3** Indentare de 3 și 5 spații (`siruri.py:29`, `:39`) — în Python folosește mereu 4.
- **C4** 15 rânduri goale la finalul lui `siruri2.py`.
- **C5** `cel_mai_scurt_cuvant("")` → `IndexError` (`siruri2.py:64`); `ascunde_mijlocul("abc")` → `'abbc'`. Încearcă-le pe `""` și pe texte scurte.

Întrebări

1. Ce întoarce `"Python"[0]`, dar `"Python"[0:1]`, dar `"Python"[::-1]`? Care e literă, care e felie?
2. La B6 și B7 bug-ul e **valoarea de pornire**. Care e regula după care alegi cu ce pornește un maxim sau un minim?
3. B1, B2 și B3 au trecut pe exemplele tale. Ce fel de exemplu ar fi trebuit să încerci, ca să le prinzi?